Système de Gestion de Tâches Collaboratives

Application web full-stack permettant la gestion de projets et de tâches en équipe, avec un système de rôles (Administrateur, Professeur, Étudiant), un chat intégré par projet, et un tableau de bord analytique.

Table des matières

- Aperçu
- Stack technique
- Architecture du projet
- Fonctionnalités
- Modèles de données
- API REST — Endpoints
- Prérequis
- Installation & Lancement
  - Backend Django
  - Frontend React
- Variables d'environnement
- Tâches automatiques (Celery)
- Tests unitaires
- Structure des dossiers

Aperçu

Ce projet est un système de gestion collaborative de tâches développé dans le cadre du cours de Programmation Web Avancée (Django 2026). Il offre :

- Un système d'inscription avec validation par l'administrateur
- La création et gestion de projets collaboratifs
- L'assignation de tâches avec suivi d'état et dates d'échéance
- Un chat en temps réel par projet
- Des dashboards personnalisés selon le rôle
- Un module de statistiques avec calcul automatique de primes pour les professeurs

---

Stack technique

Backend

Technologie	Version	Rôle
Python / Django	6.0.3	Framework web principal
Django REST Framework	3.16.1	Exposition de l'API REST
SimpleJWT	5.5.1	Authentification par tokens JWT
Celery	5.6.2	Tâches asynchrones et planifiées
Redis	7.3.0	Broker de messages pour Celery
MySQL / mysqlclient	2.2.8	Base de données relationnelle

django-cors-headers	4.9.0	Gestion des CORS
Pillow	12.1.1	Upload et gestion des photos
django-celery-beat	2.9.0	Planification des tâches Celery

Frontend

Technologie	Version	Rôle
React	19.2	Framework UI
Vite	7.3	Bundler et serveur de développement
Tailwind CSS	4.2	Framework CSS utilitaire
Redux Toolkit	2.11	Gestion de l'état global
React Router	7.13	Routage côté client
Recharts	3.7	Graphiques et visualisations
React Icons	5.5	Bibliothèque d'icônes

Architecture du projet

```
Projet_Systeme_Gestion_Tâches_Collaboratives/
├── Backend_Django/
│   ├── manage.py
│   ├── requirement.txt
│   ├── backend_app/           # Application principale
│   │   ├── models.py          # Modèles de données
│   │   ├── serializers.py     # Sérialiseurs DRF
│   │   ├── views.py           # Vues / contrôleurs
│   │   ├── urls.py            # Routes de l'API
│   │   ├── tasks.py           # Tâches Celery
│   │   └── management/
│   │       └── commands/
│   │           ├── seed.py              # Données de test
│   │           └── rappel_deadlines.py  # Rappels d'échéances
│   └── backend_project/       # Configuration Django
│       ├── settings.py
│       ├── celery.py
│       └── urls.py
└── Frontend_React/
    ├── index.html
    ├── package.json
    └── src/
        ├── pages/             # Pages par rôle
        │   ├── Administrateur/
        │   ├── Professeurs/
        │   ├── Etudiants/
```

```
        │   ├── Auth/
        │   └── Profils/
        ├── services/
        │   ├── api/           # Appels HTTP
        │   ├── hooks/         # Custom hooks React
        │   └── context/       # Contextes React
        ├── store/             # Redux store & slices
        ├── composants/        # Composants réutilisables
        └── utils/             # Fonctions utilitaires
```

Fonctionnalités

Authentification & Utilisateurs

- **Inscription** (3 étapes) : choix du rôle, informations personnelles, photo de profil
- **Validation de compte** : les nouveaux comptes sont inactifs par défaut, l'admin doit les valider
- **Connexion JWT** : tokens d'accès et de rafraîchissement
- **Gestion de profil** : modification des informations, upload/suppression de photo
- **Vérification d'email** : contrôle de disponibilité en temps réel à l'inscription
- **Matricule automatique** : généré à la création selon le rôle (`ETUDIANT-001` , `PROFESSEUR-002` , etc.)

Gestion des Projets

- Création, modification et suppression de projets
- Ajout et retrait de collaborateurs (étudiants et professeurs uniquement)
- Seul le créateur ou un administrateur peut gérer un projet
- Vue liste avec progression des tâches

Gestion des Tâches

- Création et assignation de tâches à des membres du projet
- Statuts : `À faire` / `Terminé`
- Dates d'échéance avec suivi de complétion
- Filtrage par projet ou par utilisateur connecté

Chat par Projet

- Messagerie intégrée par projet
- Compteur de messages non lus
- Marquage automatique des messages lus

Dashboards & Statistiques

Rôle	Contenu
Administrateur	KPIs globaux, validation des comptes en attente, statistiques professeurs
Professeur	Projets en cours, tâches assignées, progression
Étudiant	Tâches assignées, projets participés, avancement

Module Primes (Professeurs)

- Calcul automatique des primes mensuelles selon le taux de complétion des tâches à temps

    - 100% à l'heure → **100 000 FCFA**
    - ≥ 90% à l'heure → **30 000 FCFA**
    - < 90% → **0 FCFA**
- Leaderboard des professeurs
- Export des données par mois/année

Modèles de données

```
Utilisateur (AbstractUser)
├── email (identifiant unique)
├── nom, prenom, telephone
├── role: etudiant | professeur | administrateur
├── matricule (auto-généré)
├── promotion, photo
└── is_active (validation admin requise)

Projet
├── titre, description
├── createur → Utilisateur
└── date_creation

Collaborateur (liaison Projet ↔ Utilisateur)
├── projet → Projet
└── user → Utilisateur

Tache
├── titre
├── statut: a_faire | termine
├── projet → Projet
├── assigne_a → Utilisateur
├── date_echeance
└── date_completion

Message
├── projet → Projet
├── auteur → Utilisateur
├── contenu
└── date_envoi

DerniereLecture
├── user → Utilisateur
├── projet → Projet
└── date_lecture
```

API REST — Endpoints

Base URL : `http://localhost:8000/api/`

Authentification

Méthode	Endpoint	Description
POST	`/register/`	Créer un compte (inactif)
POST	`/login/`	Connexion → tokens JWT
POST	`/token/refresh/`	Rafraîchir le token
GET	`/me/`	Profil de l'utilisateur connecté
PATCH	`/me/photo/`	Upload photo de profil
DELETE	`/me/photo/`	Supprimer photo de profil
POST	`/check-email/`	Vérifier la disponibilité d'un email

Utilisateurs

Méthode	Endpoint	Description
GET	`/utilisateurs/`	Liste de tous les utilisateurs
POST	`/utilisateurs/create/`	Créer un utilisateur actif (admin)
GET	`/utilisateurs/count/`	Compteurs par rôle
GET	`/utilisateurs/awaitingValidation/count/`	Comptes en attente
GET/PATCH/DELETE	`/utilisateurs/<id>/`	Détail / modifier / supprimer
PATCH	`/utilisateurs/<id>/status/`	Activer / désactiver un compte

Projets

Méthode	Endpoint	Description
GET/POST	`/projets/`	Lister / créer des projets
GET/PATCH/DELETE	`/projets/<id>/`	Détail / modifier / supprimer
POST	`/projets/<id>/collaborateurs/`	Ajouter un collaborateur
DELETE	`/projets/<id>/collaborateurs/<collab_id>/`	Retirer un collaborateur

Tâches

Méthode	Endpoint	Description
GET/POST	`/taches/`	Lister / créer des tâches
GET/PATCH/DELETE	`/taches/<id>/`	Détail / modifier / supprimer

Chat

Méthode	Endpoint	Description

GET/POST	`/projets/<id>/messages/`	Messages du projet
POST	`/projets/<id>/messages/read/`	Marquer comme lu
GET	`/messages/unread/`	Compteur de non-lus

Statistiques

Méthode	Endpoint	Description
GET	`/statistiques/professeurs/`	Stats complètes des professeurs
POST	`/professeurs/eligible-for-prime/`	Calcul des primes (mois/année)
GET	`/professeur/dashboard/`	Dashboard professeur
GET	`/etudiant/dashboard/`	Dashboard étudiant

Prérequis

- Python 3.11+
- Node.js 18+ et npm
- MySQL 8+
- Redis 6+

Installation & Lancement

Backend Django

```
# 1. Aller dans le dossier backend
cd Backend_Django

# 2. Créer et activer un environnement virtuel
python -m venv venv
source venv/bin/activate      # Linux / macOS
# venv\Scripts\activate       # Windows

# 3. Installer les dépendances
pip install -r requirement.txt

# 4. Configurer la base de données (voir section Variables d'environnement)
# Créer la base MySQL : CREATE DATABASE gestion_taches CHARACTER SET utf8mb4;

# 5. Appliquer les migrations
python manage.py migrate

# 6. (Optionnel) Peupler avec des données de test
python manage.py seed

# 7. Créer un superutilisateur administrateur
python manage.py createsuperuser
```

```
# 8. Lancer le serveur Django
python manage.py runserver
```

Celery (dans un terminal séparé)

```
cd Backend_Django
source venv/bin/activate

# Worker Celery
celery -A backend_project worker --loglevel=info

# Planificateur Celery Beat (dans un autre terminal)
celery -A backend_project beat --loglevel=info
```

Frontend React

```
# 1. Aller dans le dossier frontend
cd Frontend_React

# 2. Installer les dépendances
npm install

# 3. Lancer le serveur de développement
npm run dev
```

L'application sera accessible sur `http://localhost:5173`.

Variables d'environnement

Créer un fichier `.env` dans `Backend_Django/` :

```
# Base de données MySQL
DB_NAME=gestion_taches
DB_USER=root
DB_PASSWORD=votre_mot_de_passe
DB_HOST=localhost
DB_PORT=3306

# Django
SECRET_KEY=votre_clé_secrète_django
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

# Redis / Celery
CELERY_BROKER_URL=redis://localhost:6379/0
CELERY_RESULT_BACKEND=redis://localhost:6379/0
```

> *Adapter `src/config/api.js` dans le frontend si l'URL du backend change.*

Tâches automatiques (Celery)

Tâche	Description	Fréquence
`rappel_deadlines`	Envoie des rappels pour les tâches dont l'échéance approche	Planifiable via Celery Beat

Lancer manuellement :

```
python manage.py rappel_deadlines
```

Tests unitaires

Lancer les tests

Les tests s'exécutent sur **SQLite** (base en mémoire, configurée automatiquement dans `settings.py` quand `test` est détecté dans les arguments).

```
cd Backend_Django
source env/bin/activate
python manage.py test backend_app.tests --verbosity=2
```

Couverture des tests

106 tests couvrant tous les endpoints de l'API :

Classe de test	Nb	Endpoints couverts
`RegisterViewTest`	4	`POST /api/register/`
`LoginViewTest`	4	`POST /api/login/`
`TokenRefreshTest`	1	`POST /api/token/refresh/`
`CurrentUserViewTest`	2	`GET /api/me/`
`CheckEmailViewTest`	3	`POST /api/check-email/`
`UtilisateurListViewTest`	1	`GET /api/utilisateurs/`
`UtilisateurDetailViewTest`	6	`GET/PATCH/DELETE /api/utilisateurs/<id>/`
`AdminCreateUserViewTest`	2	`POST /api/utilisateurs/create/`
`UserCountViewTest`	1	`GET /api/utilisateurs/count/`
`UsersAwaitingValidationViewTest`	1	`GET /api/utilisateurs/awaitingValidation/count/`
`UpdateUserStatusViewTest`	5	`PATCH /api/utilisateurs/<id>/status/`

`ProjetListCreateViewTest`	6	`GET/POST /api/projets/`
`ProjetDetailViewTest`	8	`GET/PATCH/DELETE /api/projets/<id>/`
`CollaborateurAddViewTest`	6	`POST /api/projets/<id>/collaborateurs/`
`CollaborateurRemoveViewTest`	3	`DELETE /api/projets/<id>/collaborateurs/<id>/`
`TacheListCreateViewTest`	7	`GET/POST /api/taches/`
`TacheDetailViewTest`	8	`GET/PATCH/DELETE /api/taches/<id>/`
`MessageListCreateViewTest`	6	`GET/POST /api/projets/<id>/messages/`
`MarkChatReadViewTest`	3	`POST /api/projets/<id>/messages/read/`
`UnreadMessagesCountViewTest`	3	`GET /api/messages/unread/`
`ProfesseurDashboardViewTest`	3	`GET /api/professeur/dashboard/`
`EtudiantDashboardViewTest`	4	`GET /api/etudiant/dashboard/`
`StatistiquesProfsViewTest`	4	`GET /api/statistiques/professeurs/`
`EligibleProfessorsForPrimeViewTest`	6	`POST /api/professeurs/eligible-for-prime/`
Tests des modèles (5 classes)	8	`Utilisateur`, `Projet`, `Tache`, `Collaborateur`, `DerniereLecture`

Scénarios testés pour chaque endpoint :

- Cas de succès (200, 201, 204)
- Cas d'erreur (400 données invalides, 404 ressource introuvable)
- Contrôle d'accès (401 non authentifié, 403 non autorisé)
- Règles métier (ex : impossible d'assigner un professeur sur un projet étudiant, un admin ne peut pas être collaborateur, etc.)

Structure des dossiers

Backend — `backend_app/`

```
models.py         → Utilisateur, Projet, Collaborateur, Tache, Message,
DerniereLecture
serializers.py    → Sérialisation DRF + JWT custom
views.py          → Toutes les vues APIView
urls.py           → Routage de l'API
tasks.py          → Tâche Celery rappel_deadlines
management/
  commands/
    seed.py               → Génération de données de test (Faker)
    rappel_deadlines.py   → Commande de rappel d'échéances
```

Frontend — `src/`

```
pages/
  Auth/               → Login (3 étapes), Register, Logout
  Administrateur/     → Dashboard, Utilisateurs, Projets, Statistiques
  Professeurs/        → Dashboard, MesProjets, MesTaches
  Etudiants/          → Dashboard, MesProjets, MesTaches
  Profils/            → Page profil commune (tous rôles)
services/
  api/                → authService, projetService, tacheService, messageService…
  hooks/              → Custom hooks par domaine (auth, projet, tache, chat…)
  context/            → ToastContext
store/
  userSlice.js        → État utilisateur connecté (Redux)
composants/           → Avatar, Badge, Sidebar, Toast, PrivateRoute…
utils/                → formatDate, formatters
```

Auteur

Projet réalisé par **Serigne Abdoulaye Babou** dans le cadre du cours **Programmation Web Avancée — Django 2026**, 1er Semestre — Projet de Fin de Cours.